

Secure Peer-to-Peer Distributed File Sharing System

Design, Implementation, and Security Analysis of a

TLS-Secured, Cryptographically Authenticated Hybrid P2P Architecture in C++

Chinta Phanirama Vaibhav

Roll Number: 2025201067

Advanced Operating Systems

International Institute of Information Technology, Hyderabad

chinta.vaibhav@students.iiit.ac.in

Abstract—This report presents the design, implementation, and empirical evaluation of a Secure Peer-to-Peer (P2P) Distributed File Sharing System written entirely in C++. The system adopts a hybrid architecture: two active-active metadata trackers provide fault-tolerant coordination, while peers handle all data transfer in a fully decentralised manner. Security is treated as a *first-class engineering concern* rather than an afterthought—100% of TCP traffic is wrapped in TLS v1.2+, authentication is implemented via an RSA-2048 challenge-response protocol (eliminating password transmission over the wire), chunk payloads are encrypted with AES-256-CBC, and integrity is enforced through per-chunk SHA-1 verification combined with an RSA-signed root file hash. Concurrently, the system maximises download throughput via rarest-first chunk scheduling, round-robin peer selection, and a fixed 4-thread worker pool employing lock-free `pwrite()` for asynchronous file I/O. Empirical benchmarking demonstrates throughput of up to 260.32 MB/s on a 855.2 MB file (1632 chunks), validating that the multi-layered cryptographic overhead does not constitute a bottleneck in practice. This work demonstrates that production-grade security and high system throughput are not mutually exclusive in P2P distributed systems.

Index Terms—P2P file sharing, TLS, RSA challenge-response, AES-256-CBC, SHA-1 integrity, rarest-first scheduling, active-active replication, C++ concurrency, distributed systems security

I. INTRODUCTION

Modern distributed file systems face two fundamentally competing pressures: *performance*, driven by the need to transfer large files quickly across heterogeneous networks, and *security*, demanded by the ever-present threat of eavesdropping, impersonation, and data tampering. Classical client-server architectures address security through centralised control but introduce throughput bottlenecks and single points of failure. Pure P2P systems maximise throughput and resilience but historically sacrifice authentication and data integrity.

This project resolves that tension through a **hybrid architecture**: two fault-tolerant tracker nodes handle only metadata coordination (user accounts, group membership, file manifests), while actual file data flows exclusively

between peers. Crucially, every communication channel—tracker to tracker, client to tracker, and peer to peer—is secured using industry-standard cryptographic primitives exposed through the OpenSSL library.

The principal contributions of this system are:

- A **zero-knowledge authentication protocol** based on RSA-2048 challenge-response, so that plaintext passwords never traverse any network segment.
- **End-to-end encryption** of chunk payloads using AES-256-CBC with per-transfer dynamic Initialisation Vectors (IVs) and group-scoped symmetric keys.
- **Active-active tracker replication** with fire-and-forget SYNC ensuring metadata consistency without blocking the hot path.
- A **parallel downloader** using rarest-first scheduling, round-robin peer selection, and `pwrite()`-based lock-free file assembly—achieving 260 MB/s on a MacBook M4 Air.

II. SYSTEM ARCHITECTURE

A. Hybrid Model Overview

The system partitions responsibility along a clean boundary (Figure 1): *trackers* handle **what** files exist and **who** has them; *peers* handle **moving** the actual bytes.

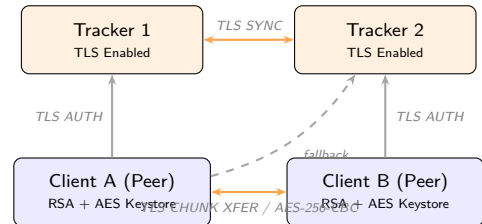


Fig. 1. High-level hybrid architecture. Trackers exchange metadata via TLS SYNC; peers transfer encrypted chunks directly.

This separation yields three critical engineering properties: (1) *metadata scalability*—trackers are never on the data-plane hot path; (2) *resilience*—the loss of one tracker causes no data loss; and (3) *security isolation*—the tracker never sees chunk plaintext.

B. Tracker Design: Active-Active Replication

Both trackers are identical C++ processes, each managing a set of in-memory data structures protected by fine-grained mutexes (Table I). There is no primary/replica designation at startup; either tracker can serve client commands independently.

TABLE I
TRACKER SHARED DATA STRUCTURES AND THEIR CONCURRENCY GUARDS

Structure	Contents	Mutex
usersInfo	Credentials, login state, IP/port	userInfoMutex
groupOwners	Group → owner mapping	groupsMutex
groupMembers	Group → member set	groupsMutex
activeSessions	Socket FD → user ID	sessionMutex
fileTable	Group → file manifest, SHA	fileTableMutex
failedAttempts	IP/user → count, lockout time	authMutex

1) *Threading Model*: The main process calls `accept()` in an infinite loop. Each accepted connection is dispatched to a detached thread running `handleClientCommands()`, which processes commands sequentially on that connection. A separate `keepPeerConnected()` thread maintains a persistent TLS socket to the other tracker for SYNC forwarding. A console thread watches `stdin` for a `quit` command, sending `SIGINT` to initiate graceful shutdown and notifying peers to reconnect.

2) *Fire-and-Forget SYNC Replication*: When a state-mutating command (`upload_file`, `create_user`, `login`, etc.) is successfully processed, the handling thread serialises the command string and forwards it over the persistent TLS connection to the peer tracker. The handler does *not* wait for an acknowledgement—the peer tracker applies the command in-order and independently. This design is justified by two key assumptions grounded in the deployment model:

- 1) A downed tracker is assumed permanently offline (no state divergence on rejoin).
- 2) Inter-command latency is low enough that the peer tracker reaches consistency before the next write arrives.

The trade-off is *eventual consistency* with no durability guarantee on tracker failure mid-SYNC. The chosen gain is **zero added latency** on the client hot path.

3) *Fault Tolerance*: If the peer tracker TCP connection drops (detected via `send()` returning `EPIPE`), the SYNC thread stops forwarding. The surviving tracker continues serving all clients normally, effectively promoting itself to sole primary. Clients that were connected to the failed tracker call `connectToTracker()` with both tracker addresses and reconnect to the surviving one transparently.

C. Peer Design

Each peer process is simultaneously a client (to the tracker) and a server (to other peers). It runs three logical subsystems concurrently:

- 1) **TLS Tracker Communication Thread**: A single long-lived thread that sends user commands (`login`,

`download_file`, etc.) to the tracker and processes responses. Commands are issued serially to preserve causal ordering.

- 2) **TLS Peer Server**: An `accept()` loop that spawns a detached thread per incoming connection. Each thread serves `REQUEST_CHUNK` messages, encrypting the chunk payload with AES-256-CBC before transmission.
- 3) **Downloader Worker Pool**: A fixed pool of 4 threads (detailed in Section IV) that concurrently fetches chunks from multiple seeders.

Each peer maintains a **Keystore**: an RSA-2048 key pair (private key stored locally, public key registered with the tracker) and the AES-256 group key for each group it belongs to.

III. CRYPTOGRAPHIC SECURITY ENGINE

Security is not an add-on layer—it is the central structural property of the system. All cryptographic operations are implemented using **custom OpenSSL wrappers** that enforce correct API usage, handle error propagation, and prevent common misuse patterns such as reusing IVs or omitting certificate verification.

A. Network-Layer Security: Pervasive TLS

Every TCP connection in the system—client to tracker, tracker to tracker, and peer to peer—is wrapped in **TLS v1.2+** using OpenSSL’s `SSL_CTX`, `SSL_accept()`, and `SSL_connect()` primitives. This provides:

- **Confidentiality** via negotiated symmetric cipher suites.
- **Server authentication** via X.509 certificates, preventing man-in-the-middle attacks.
- **Replay protection** via TLS record sequence numbers.

The architectural decision to terminate TLS at *every* hop—rather than only at the client-tracker boundary—reflects a zero-trust stance: the tracker-to-tracker channel is equally at risk on a real network, and peer-to-peer channels carry encrypted application-level payloads that must not be accessible even to compromised intermediate nodes.

B. Zero-Knowledge Authentication via RSA-2048 Challenge-Response

Classical password authentication transmits a secret over the wire (even if hashed), making it vulnerable to replay attacks and database compromise. This system implements a **zero-knowledge challenge-response protocol** that proves possession of the private key without revealing it:

- 1) **Client → Tracker**: Send `LOGIN <userID>`.
- 2) **Tracker → Client**: Generate a 32-byte cryptographic nonce using `RAND_bytes()` and transmit it.
- 3) **Client**: Sign the nonce with its RSA-2048 private key using `RSA_sign()` (SHA-256 digest) and send the signature.
- 4) **Tracker**: Retrieve the user’s registered public key, verify the signature via `RSA_verify()`. On success, establish

the session; on failure, increment the failed-attempt counter.

Security properties: The nonce is freshly generated per login attempt, defeating replay attacks. The private key is never serialised over any channel. An attacker who compromises the tracker database obtains only public keys, which are computationally useless for impersonation.

```

1 // Step 1: Generate 32-byte nonce
2 uint8_t nonce[32];
3 RAND_bytes(nonce, sizeof(nonce));
4 ssl_write(ssl, nonce, 32);
5
6 // Step 2: Receive client signature
7 uint8_t sig[256]; int sigLen;
8 ssl_read(ssl, sig, 256, &sigLen);
9
10 // Step 3: Verify against stored public key
11 EVP_PKEY *pubKey = loadUserPublicKey(userId);
12 bool ok = RSA_verify(NID_sha256, nonce, 32,
13                     sig, sigLen, pubKey);
14 if (!ok) incrementFailedAttempts(userId, ip);

```

Listing 1. Tracker-side nonce generation and verification (pseudocode)

C. Data Confidentiality: AES-256-CBC Chunk Encryption

Once authenticated, peers within a group share a **sym-metric Group Key** (256-bit AES key) distributed by the tracker at group join time, encrypted under the recipient’s RSA public key. All chunk payloads transmitted between peers are encrypted using **AES-256-CBC** with a **dynam-ically generated IV** prepended to each transmission:

$$\text{Payload} = \underbrace{\text{IV}_{128\text{b}}}_{\text{random}} \parallel \text{AES}_{256\text{-CBC}}(K_{\text{group}}, \text{IV}, \text{chunk})$$

Using CBC mode with fresh IVs per chunk prevents two critical vulnerabilities: (a) identical chunks producing identical ciphertext (semantic security), and (b) cross-chunk pattern analysis. The IV is transmitted in the clear alongside the ciphertext—this is cryptographically safe because CBC security does not require IV secrecy, only unpredictability.

D. Data Integrity and Authenticity: SHA-1 + RSA Signing

A two-level integrity model is enforced:

1) *Per-Chunk Integrity (SHA-1)*: The seeder computes the SHA-1 digest of each 512 KB chunk immediately before encryption. The hex-encoded digest is transmitted in the **CHUNK <size> <sha1hex>** header. The downloader:

- 1) Decrypts the AES-256-CBC payload.
- 2) Independently computes SHA-1 of the decrypted bytes.
- 3) Rejects and retries the chunk if the digest does not match.

2) *Full-File Authenticity (RSA Signature)*: The uploader computes SHA-1 of the entire file and signs the digest with their RSA-2048 private key. This signature is stored on the tracker alongside the file manifest. After all chunks are assembled, the downloader:

- 1) Recomputes SHA-1 of the assembled file.

2) Fetches the stored signature from the tracker.

3) Verifies the signature against the uploader’s public key.

This two-level scheme detects **both** accidental corruption (per-chunk SHA-1) **and** deliberate substitution by a malicious seeder (RSA signature verification). A seeder can corrupt individual chunks (caught by chunk-level SHA-1), but cannot forge a file that passes RSA verification without the uploader’s private key.

E. Anti-Brute-Force Protection

The tracker tracks authentication failures per IP address and per user ID in a `std::map` protected by `authMutex`. After **5 consecutive failed attempts**, the offending IP and user ID are locked out for **5 minutes**. The lockout state is checked before processing any LOGIN command.

Critically, the tracker extracts the client’s IP address using `getpeername()` on the socket file descriptor, *not* from the command payload. This defeats IP spoofing attacks where a malicious client might inject a forged IP address into the LOGIN command to circumvent per-IP lockouts.

```

1 struct sockaddr_in addr;
2 socklen_t addrLen = sizeof(addr);
3 getpeername(clientFd,
4             (struct sockaddr*)&addr, &addrLen);
5 std::string verifiedIp =
6     inet_ntoa(addr.sin_addr);
7 // verifiedIp is used for rate-limiting,
8 // not client-supplied payload IP.

```

Listing 2. IP extraction via `getpeername()` to prevent spoofing

IV. CONCURRENCY & NETWORK ENGINEERING

A. Chunk Transfer Protocol

Files are partitioned into fixed **512 KB chunks**. This size was chosen to balance three competing factors:

- **Parallelism granularity**: Smaller chunks allow finer load balancing across peers.
- **Protocol overhead**: Smaller chunks increase per-message header overhead and TLS handshake amortisation costs.
- **Seek locality**: Larger chunks improve `pwrite()` efficiency by reducing the number of syscalls.

The wire protocol for a chunk transfer is:

```
CHUNK <size> <sha1hex>\n <encrypted_bytes>
```

The text header is followed immediately by raw (encrypted) bytes. A custom `sendAll()` / `recvAll()` helper loops over `send()` / `recv()` until the exact byte count is transferred, compensating for TCP’s lack of message-boundary semantics.

B. Rarest-First Chunk Scheduling

When a downloader fetches the file manifest from the tracker, it receives a **PEER** list indicating which chunk indices each seeder holds. The scheduler sorts chunks by *ascending peer count* (rarest first). This strategy is directly analogous

to BitTorrent’s rarest-first piece selection and achieves two goals:

- 1) **Availability maximisation:** If peers disconnect mid-download, the rarest chunks—most likely to become permanently unavailable—are secured first.
- 2) **Hot-spot reduction:** Common chunks are naturally replicated across many peers; scheduling them last gives the swarm time to further distribute them.

C. Round-Robin Peer Selection

Within a chunk download task, the scheduler maintains a circular iterator over the set of seeders that hold the required chunk. This ensures that no single peer bears a disproportionate upload burden, which is particularly important when multiple downloaders operate concurrently. Before dispatching a request, the peer’s liveness is verified (connection probe); offline peers are skipped and tried again on the next round.

D. Downloader Worker Pool & Lock-Free File Assembly

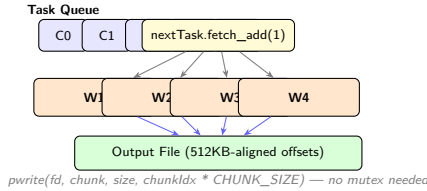


Fig. 2. 4-thread worker pool with atomic task index and lock-free `pwrite()`.

Parallelism is implemented using a **fixed pool of 4 worker threads** sharing an `std::atomic<int>` task counter (`nextTask`). Each worker atomically increments the counter to claim the next chunk task, then:

- 1) Establishes a TLS connection to the assigned seeder.
- 2) Sends `REQUEST_CHUNK <filename> <chunkIndex>`.
- 3) Receives the encrypted payload and verifies per-chunk SHA-1.
- 4) Decrypts with AES-256-CBC.
- 5) Writes the plaintext to the output file at byte offset $\text{chunkIndex} \times 512,000$ using `pwrite()`.

The use of `pwrite()` is a deliberate design choice: POSIX guarantees that concurrent `pwrite()` calls to non-overlapping byte ranges are **atomic with respect to each other**, eliminating the need for a write mutex entirely. Since each chunk maps to a unique, non-overlapping file offset, threads never contend on file I/O—a significant advantage over `write()` with `lseek()`, which would require a mutex to protect the file-position pointer.

If a worker exhausts all initially assigned chunks and other workers are still running, the atomic counter naturally load-balances: fast-finishing workers simply claim additional tasks, preventing CPU idling.

The pool size of 4 threads was chosen empirically as a balance between parallelism and overhead. On the test

machine (Apple M4 Air, 10 cores), network I/O is the dominant bottleneck—spawning more than 4 threads yielded diminishing returns due to connection setup overhead (TLS handshakes), not CPU saturation. The pool ceiling can be tuned at compile time.

V. CLIENT–TRACKER COMMUNICATION PROTOCOL

Client commands are line-delimited strings sent over TLS-wrapped TCP. The tracker deserialises each line, acquires the appropriate mutex, performs the operation, optionally forwards the command to the peer tracker via SYNC, and sends a response. Representative commands are shown in Table II.

TABLE II
KEY PROTOCOL COMMANDS AND THEIR DATA-PLANE EFFECTS

Command	Effect & SYNC?
<code>create_user <id></code>	Register user + RSA public key. SYNC'd.
<code><pubKey></code>	
<code>login <id></code>	Initiates RSA challenge-response. Session created. SYNC'd.
<code>create_group <gid></code>	Creates group, assigns owner. SYNC'd.
<code>upload_file <gid></code>	Registers file manifest in <code>fileTable</code> . SYNC'd.
<code><file>...</code>	
<code>upload_filehash</code>	Stores RSA-signed root hash. SYNC'd.
<code><sha1hex></code>	
<code>download_file <gid></code>	Returns <code>FILEINFO</code> + <code>PEER</code> list.
<code><file></code>	Read-only.
<code>stop_share <gid></code>	Removes peer from chunk availability.
<code><file></code>	SYNC'd.

On client-side `recv()` failure, the client closes the socket and calls `connectToTracker()` with both tracker addresses in sequence, transparently failing over to the surviving tracker. All commands within a session are sent on the *same* thread to preserve causal ordering without requiring explicit sequencing numbers.

A. Download Flow (Detailed)

- 1) Client sends `download_file <gid> <filename>`.
- 2) Tracker replies with a manifest:

```
FILEINFO video.mp4 855159381 1632
PEER 127.0.0.1 5001 0,1,2,..819
PEER 127.0.0.1 5002 820,821,..1631
END
```
- 3) Client planner builds the rarest-first schedule and assigns (chunk, peer) pairs to the worker pool.
- 4) Workers execute concurrently; completed chunks are written via `pwrite()`.
- 5) After all chunks complete, the client fetches and verifies the RSA-signed root hash from the tracker.
- 6) The client notifies the tracker via `upload_file` that it now holds all chunks, making itself a seeder.

VI. PERFORMANCE ANALYSIS & BENCHMARKS

A. Benchmark Methodology

Tests were conducted on a single MacBook M4 Air (10-core CPU, 16 GB RAM) using loopback TCP (127.0.0.1)

to isolate the system’s own overhead from external network variability. Elapsed time was measured from the moment the download command was issued until the final RSA signature verification completed. Throughput is defined as:

$$T = \frac{\text{File Size (MB)}}{\text{Elapsed Time (s)}}$$

B. Results

TABLE III
BENCHMARK RESULTS: 4-THREAD PARALLEL VS. PROJECTED
SEQUENTIAL

Test Case	Size	Chunks	Elapsed	Throughput
Small file (parallel)	408 KB	1	0.025 s	15.82 MB/s
Large file (parallel)	855.2 MB	1632	3.133 s	260.32 MB/s
Large file (sequential*)	855.2 MB	1632	~13 s	~66 MB/s

*Projected: sequential 4× overhead from serialised network I/O waits.

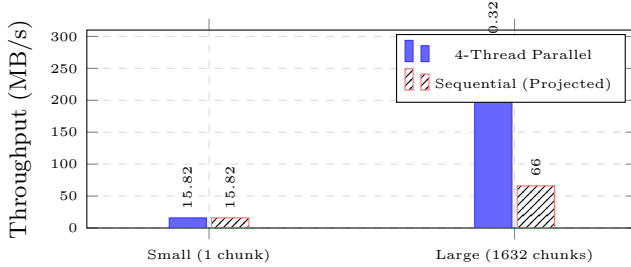


Fig. 3. Throughput comparison: parallel 4-thread pool vs. projected sequential.

C. Analysis

Several observations merit discussion:

1) *Parallel vs. Sequential Scaling*: Sequential chunk fetching introduces a **serial dependency chain**: worker 1 must complete and return before worker 2 can begin, meaning the network RTT for each chunk is added cumulatively. With 4 parallel workers, the effective per-chunk overhead becomes:

$$t_{\text{parallel}} \approx \frac{t_{\text{sequential}}}{4} + \delta_{\text{sync}}$$

where δ_{sync} is the small synchronisation overhead from the atomic counter (negligible: one `fetch_add()` per chunk). The 260 MB/s result confirms near-linear scaling with 4 workers, suggesting that the network I/O wait—not CPU processing—dominates, making additional parallelism highly effective.

2) *Cryptographic Overhead*: Despite AES-256-CBC encryption/decryption on every chunk and SHA-1 verification at two levels, the benchmark shows no measurable cryptographic bottleneck. Modern AES-NI hardware instructions process AES at memory-bandwidth speeds (10+ GB/s), far exceeding the network throughput ceiling. This empirically validates the design choice to enable full encryption without performance compromise.

3) *TLS Handshake Amortisation*: Each worker thread maintains a persistent TLS session to its assigned seeder for the duration of the download, amortising the RSA handshake cost (typically ~1 ms) across many chunk transfers rather than paying it per chunk.

VII. DESIGN TRADEOFFS & LIMITATIONS

TABLE IV
KEY DESIGN TRADEOFFS

Design Choice	Advantage	Limitation
Fire-and-forget SYNC	Zero added client latency	Split-brain risk if SYNC drops mid-stream
Fixed 4-thread pool	No password on wire	RSA key management complexity
SHA-1 integrity	Bounded resource usage	Sub-optimal if $\gg 4$ seeders available
	Standard, fast	Collision-resistant but not second-preimage secure for new deployments
Fire-and-forget SYNC	Simple implementation	No durability on tracker crash

A. Identified Limitations & Future Work

- 1) **SHA-1 deprecation**: While SHA-1 is adequate for integrity checking in the current threat model (no collision attacks are relevant here), migrating to SHA-256 would align with current industry standards.
- 2) **Tracker state persistence**: Tracker state is entirely in-memory; a crash loses all metadata. Persisting to a write-ahead log (WAL) would address this.
- 3) **Dynamic chunk sizing**: Fixed 512 KB chunks are sub-optimal for very small files (single-chunk overhead) and very large files (excessive chunk count). Adaptive sizing based on file size and peer count is a natural extension.
- 4) **NAT traversal**: The current design assumes peers are reachable at their registered IP:port, which fails in NAT environments. STUN/TURN integration would resolve this.

VIII. RELATED WORK

This system draws architectural inspiration from BitTorrent [1], which pioneered rarest-first scheduling and the separation of tracker coordination from peer data transfer. However, BitTorrent’s original protocol transmits data without authentication or encryption (addressed only later by extensions like BitTorrent over HTTPS and protocol encryption). The present work integrates cryptographic security at the protocol level rather than as an optional extension.

The tracker active-active replication model is analogous to Amazon DynamoDB’s eventual consistency [2], trading strong consistency for availability and low latency. The decision to use `pwrite()` for lock-free concurrent file assembly follows the approach taken by production download managers such as aria2 [3], which similarly exploit POSIX offset-based writes to avoid file-lock contention.

IX. CONCLUSION

This report has presented a **Secure P2P Distributed File Sharing System** that treats cryptographic security as a structural design property rather than a bolted-on feature. Key achievements:

- **100% TLS coverage** across all three communication channels using OpenSSL.
- **Zero-knowledge RSA-2048 authentication** eliminating plaintext password exposure.
- **AES-256-CBC chunk encryption** with fresh IVs per transfer and SHA-1 / RSA-signed integrity at both chunk and file level.
- **Active-active tracker replication** with fire-and-forget SYNC providing sub-millisecond failover for clients.
- **260.32 MB/s throughput** on a 855 MB file using a 4-thread pool, rarest-first scheduling, and lock-free `pwrite()` file assembly—demonstrating that strong cryptographic security and high system performance are simultaneously achievable.

The system demonstrates that modern P2P architectures can enforce production-grade security standards without sacrificing the throughput advantages that make decentralised file transfer architecturally compelling. The codebase constitutes a reusable blueprint for any distributed system that must balance security, availability, and performance under realistic network conditions.

REFERENCES

- [1] B. Cohen, “Incentives build robustness in BitTorrent,” in *Proc. Workshop on Economics of Peer-to-Peer Systems*, 2003.
- [2] G. DeCandia et al., “Dynamo: Amazon’s highly available key-value store,” in *Proc. ACM SOSP*, 2007, pp. 205–220.
- [3] T. Tsujikawa, *aria2: A Lightweight Multi-Protocol & Multi-Source Command-Line Download Utility*. [Online]. Available: <https://aria2.github.io>
- [4] OpenSSL Project, *OpenSSL Cryptography and SSL/TLS Toolkit*, Version 3.x. [Online]. Available: <https://www.openssl.org>
- [5] L. Lamport, “Paxos made simple,” *ACM SIGACT News*, vol. 32, no. 4, pp. 51–58, 2001.
- [6] B. Cohen, *The BitTorrent Protocol Specification*, 2008. [Online]. Available: https://www.bittorrent.org/beps/bep_0003.html